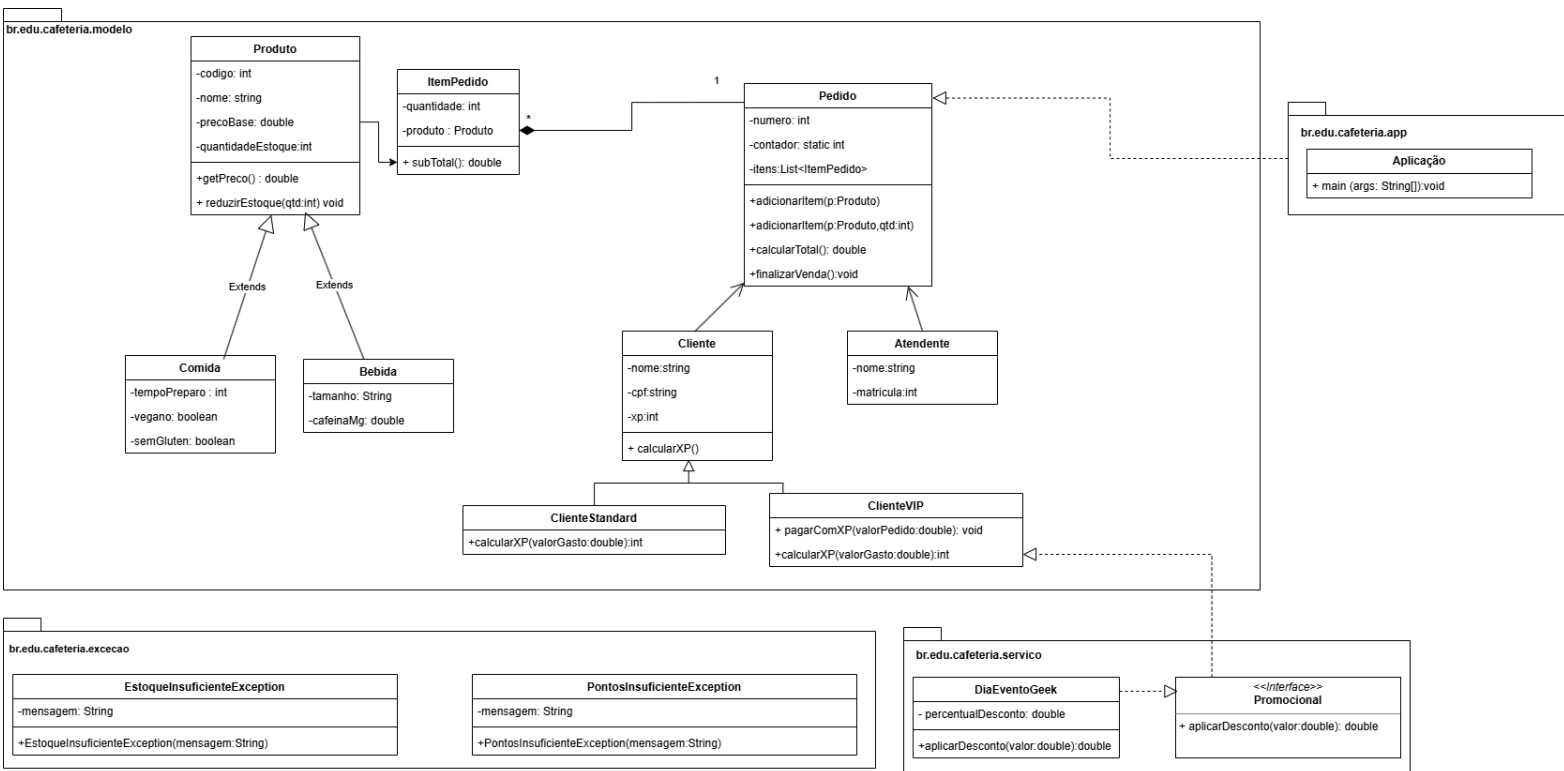


DIAGRAMA (UML)



Relatório – Sistema de Vendas e Fidelidade "Byte & Brew"

1. Associações, Heranças e Polimorfismos Aplicados

Associações

A classe **Pedido** possui associação com a classe **Cliente**, pois um pedido pode estar vinculado a um cliente cadastrado ou ser realizado por um cliente casual. Também possui associação com a classe **Atendente**, responsável pelo registro da venda.

Além disso, a classe **Pedido** tem associação com **ItemPedido**, que representa cada produto incluído na compra juntamente com sua quantidade. Cada objeto **ItemPedido** está associado a um objeto **Produto**, permitindo que um mesmo produto esteja presente em diferentes pedidos.

Herança

A classe **Produto** foi definida como **superclasse**, contendo os atributos e comportamentos comuns a todos os produtos da cafeteria, como código, nome, preço e quantidade em estoque.

As classes **Comida** e **Bebida** herdam de **Produto**, adicionando apenas as informações específicas de cada tipo.

A classe **Cliente** foi utilizada como superclasse para representar clientes cadastrados. As classes **ClienteStandard** e **ClienteVIP** herdam seus atributos básicos (nome, CPF e saldo de XP), implementando regras diferentes para o cálculo e utilização dos pontos de fidelidade.

Polimorfismo por Sobrescrita (Override):

As subclasses **ClienteStandard** e **ClienteVIP** sobrescrevem o método responsável pelo cálculo dos pontos de experiência (XP). Cada tipo de cliente possui uma regra diferente para acumular pontos, permitindo que o mesmo método apresente comportamentos distintos conforme o tipo do objeto.

Polimorfismo por Sobrecarga (Overload):

Na classe **Pedido**, o método **adicionarItem()** foi implementado com duas assinaturas:

```
adicionarItem(Produto produto)  
adicionarItem(Produto produto, int quantidade)
```

permitindo adicionar um único produto ou informar explicitamente a quantidade desejada.

Polimorfismo por Inclusão (Subtipagem):

A classe **Pedido** trabalha com objetos do tipo **Produto**, podendo armazenar tanto objetos da classe **Comida** quanto da classe **Bebida**, aproveitando o comportamento comum definido na superclasse.

Polimorfismo por Coerção:

Há conversão automática de tipos durante operações envolvendo valores inteiros e reais, como nos cálculos de preço, descontos e pontuação de XP, onde valores do tipo `int` são convertidos para `double` quando necessário.

Justificativa para as Exceções Customizadas

EstoqueInsuficienteException

Essa exceção é lançada quando o usuário tenta adicionar ao pedido uma quantidade de produtos superior à disponível em estoque.

Sua utilização impede que o sistema realize vendas impossíveis de serem atendidas, garantindo a consistência dos dados e evitando que o estoque fique negativo.

PontosInsuficientesException

Essa exceção é utilizada quando um **ClienteVIP** tenta pagar um pedido utilizando seus pontos de XP, mas não possui saldo suficiente para cobrir o valor da compra.

Essa validação garante que o programa de fidelidade siga corretamente as regras estabelecidas pela cafeteria, impedindo resgates inválidos.

Justificativa

A utilização de exceções customizadas torna o código mais organizado, facilita a identificação dos erros relacionados às regras de negócio e além disso, melhora a manutenção do sistema, pois cada situação excepcional possui uma classe própria e um tratamento específico.